

Bài tập B-Tree

1. Trình bày định nghĩa và các tính chất của cây B-Tree.

- Định nghĩa của cây B-Tree:

- B-tree là cây cân bằng được thiết kế để làm việc tốt trên đĩa từ hay thiết bị lưu trữ thứ cấp truy xuất trực tiếp khác (direct-access secondary storage device).
- Cây B tổng quát hóa cây tìm kiếm nhị phân theo cách tự nhiên.

- Các tính chất của cây B-Tree:

- (i) Tất cả các node lá cùng mức.
- (ii) Tất cả các node trung gian (trừ node gốc) có nhiều nhất m cây con và có ít nhất $m/2$ cây con (khác rỗng) (m là bậc của B-Tree).
- (iii) Mỗi node hoặc là node lá hoặc có $k+1$ cây con (k là số khoá của node này).
- (iv) Node gốc có nhiều nhất m cây con hoặc có thể có 2 cây con (Node gốc có 1 khoá và không phải là node lá) hoặc không chứa cây con nào (node gốc có 1 khoá và cũng là node lá).

2. Cài đặt tất cả các thao tác trên cây B-Tree.

- Cấu trúc dữ liệu cho B-Tree:

```
class BTree_node{  
  
    vector<my_data> keys;  
  
    vector<BTree_node*> children;  
  
    bool is_leaf(){ return children.empty(); }  
  
    friend class BTree;  
  
};  
  
class BTree{  
  
public:  
  
    BTree(int initializing_m){  
  
        if (initializing_m %2 == 0) {
```

```
initializing_m = initializing_m/2 + 1; }
```

```
this->_m = initializing_m;
```

```
root = NULL;}
```

```
bool empty(){
```

```
    return root == NULL;}
```

```
private:
```

```
BTree_node *root;
```

```
int _m;
```

```
- Searching:
```

```
pair<int, vector<BTree_node*>> search(BTree_node*root, my_data key){
```

```
    vector<BTree_node*> path;
```

```
    auto p = root;
```

```
    while(p != NULL){
```

```
        path.push_back(p);
```

```
        auto i = lower_bound(p->keys.begin(), p->keys.end(), key);
```

```
        if (i != p->keys.end() && *i == key){
```

```
            return {i - p->keys.begin(), path}; ///FOUND
```

```
        } else {
```

```
            if (p->is_leaf() == 0) break;
```

```
            p = p->children[i - p->keys.begin()];
```

```
        }
```

```
    }
```

```
    return {-1, path};
```

```
}
```

- Node splitting:

```
BTree_node* split(BTree_node *i, BTree_node*p){  
    if(i->keys.size() < this->_m){          //No need to split  
        return NULL;  
    }  
  
    //Create new BTree_node  
    auto n = new BTree_node;  
  
    auto move_up = i->keys[i->keys.size()/2];  
  
    move(i->keys.begin() + i->keys.size()/2 + 1, i->keys.end(), back_inserter(n->keys));  
  
    if ( !i->is_leaf() ) {  
        move(i->children.begin() + i->keys.size()/2+1, i->children.end(), back_inserter(n->children));  
    }  
  
    if (!i->is_leaf())        i->children.resize(i->keys.size()/2+1);  
  
    i->keys.resize(i->keys.size()/2);  
  
    if (p == NULL){          // if i has no parent  
        p = new BTree_node;  
        p->children.emplace_back(i);  
    }  
  
    int ip = lower_bound(p->keys.begin(), p->keys.end(), move_up) - p->keys.begin();  
  
    p->keys.insert(p->keys.begin() + ip, move_up);  
  
    p->children.insert(p->children.begin() + ip + 1, n);  
  
    return p;  
}
```

```

    }
}

BTree_node* split(vector<BTree_node*> &path){

    auto i = path.end() - 1;

    BTree_node*n;

    int move_up;

    while(i != path.begin() - 1){

        auto p = i - 1;

        if (i == path.begin()){

            auto x = split(*i, NULL);

            if (x == NULL) return path.front();

            else return x;

        } else {

            auto x = split(*i, *p);

            if (x == NULL) return path.front();

            else {

                i = p; // Move up the path

            }

        }

    }

}

```

- Catenation:

```

int find_cat_partner(BTree_node* p, int ip){

```

```

    int s = p->children[ip]->keys.size();

    if (ip > 0 && p->children[ip - 1]->keys.size() + s < this->_m - 1)

        return ip - 1;

    if (ip < p->children.size()-1 && p->children[ip+1]->keys.size() + s < this->_m - 1)

        return ip;

    return -1;

}

my_data catenation(BTree_node* p, int icat){

    auto moved_key = p->keys[icat];

    p->children[icat]->keys.emplace_back(p->keys[icat]);

    move(p->children[icat+1]->keys.begin()

        , p->children[icat+1]->keys.end()

        , back_inserter(p->children[icat]->keys));

    move(p->children[icat+1]->children.begin()

        , p->children[icat+1]->children.end()

        , back_inserter(p->children[icat]->children));

    delete(p->children[icat+1]);

    p->keys.erase(p->keys.begin()+icat);

    p->children.erase(p->children.begin()+icat+1);

    return moved_key;

}

```

- Deletion:

```

my_data find_replace_and_swap(int &idx, vector<BTree_node*>&path){

```

```

auto x = path.back();

auto i = path.back()->children[idx];

while(!i->is_leaf()){

    path.push_back(i);

    i = i->children.back();

}

path.push_back(i);

auto back = i->keys.back();

swap(x->keys[idx], i->keys.back());

idx = i->keys.size() - 1;

return back;

}

BTree_node* check_for_underflow_and_cat(my_data remove_key, vector<BTree_node*>
    path){

    auto i = path.end() - 1;

    auto p = i - 1;

    while(i != path.begin()){

        int ip = lower_bound((*p)->keys.begin(), (*p)->keys.end(), remove_key)

        - (*p)->keys.begin();

        int iu = underflow_check(*p, ip);

        if (iu != -1){

            //If we can underflow, we do underflow first

            underflow(*p, iu);

            return *(path.begin());

```

```

    }

    //we check for catenation

    int icat = find_cat_partner(*p, ip);

    if (icat != -1){

        //The catenation propagate

        auto cat = catenation(*p, icat);

        i = p; p--;

        remove_key = cat ;

    } else {

        return *(path.begin());

    }

}

if((*i)->keys.size() == 0){

    // The last key in root was remove

    if ((*i)->children.size() != 0){

        auto p = (*i)->children.front();

        delete *i; return p;

    }

    delete *i;

    return NULL;

} else {

    return (*i);

}

```

}

3. Cho B-tree bậc 3 gồm các khóa sau (chèn vào theo thứ tự): 81, 62, 69, 21, 75, 95, 61, 9, 57, 2, 96, 52, 53, 6, 5, 18, 82, 46. (Lưu ý thực hiện từng bước làm)

a. Các khóa nào khi thêm vào sẽ làm phát sinh thao tác split node? Vẽ cây B-Tree trước và sau khi thêm các khóa trên.

	B-Tree trước khi thêm	B-Tree sau khi thêm
- 69		
- 95		
- 61		
- 57		
- 53		
- 6		
- 82		

b. Thực hiện xóa các node theo thứ tự: 53, 69, 81

	B-Tree trước khi xóa	B-Tree sau khi xóa
- 53		
- 69		
- 81		

4. Cho B-tree bậc 5 gồm các khóa sau (chèn vào theo thứ tự): 74, 29, 46, 22, 10, 58, 80, 90, 38, 37, 37, 37, 44, 93, 74, 17, 19, 33, 1, 12, 83, 48, 84, 35, 7, 59, 55, 74, 50, 29, 12, 91, 46.

a. Các khóa nào khi thêm vào sẽ làm phát sinh thao tác split node? Vẽ cây B-Tree trước và sau khi thêm các khóa trên.

	B-Tree trước khi thêm	B-Tree sau khi thêm
- 10		
- 90		

- 44		
- 1		
- 84		
- 55		

b. Thực hiện xóa các node theo thứ tự: 33, 12, 84, 29

	B-Tree trước khi xóa	B-Tree sau khi xóa
- 33		
- 12		
- 84		
- 29		

Cách nộp: Chụp hình bài làm hoặc làm vô file word (dùng phần mềm vẽ cây)

5. Cho B-tree bậc 5 gồm các khóa sau (chèn vào theo thứ tự): 3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56

Thực hiện các yêu cầu sau:

- Thêm các khóa: 2, 6, 12

	B-Tree trước khi thêm	B-Tree sau khi thêm
- 2		
- 6		
- 12		

- Xóa khóa: 4, 5, 7, 3, 14

	B-Tree trước khi xóa	B-Tree sau khi xóa
- 4		
- 5		
- 7		
- 3		
- 14		

5. Khởi tạo B Tree bậc 7 với các thao tác Insert: 34, 12, 55, 21, 6, 84, 5, 33, 15, 74, 54, 28, 10, 19. Hãy thực hiện các chuỗi thao tác sau và cho biết kết quả qua từng thao tác:

- Insert(11)

- Delete(15)
- Delete(6)
- Insert(98)
- Delete(34)
- Delete(5)

	B-Tree trước khi thực hiện	B-Tree sau khi thực hiện
Insert(11)		
Delete(15)		
Delete(6)		
Insert(98)		
Delete(34)		
Delete(5)		